

Student

I keep hearing window functions are different from GROUP BY, but every example I see looks similar - both seem to compute something 'per group.' What is actually different?

ChatGPT • AI Assistant

CONCEPT CARD

This is the single most important distinction to understand before writing a single window function: **GROUP BY collapses rows. A window function does NOT.**

- **GROUP BY** takes many rows and produces **one summary row per group** - the individual rows are gone from the output, replaced by an aggregate.
- A **window function** computes a calculation across a set of related rows, but **every original row survives in the output**, with the computed value attached as a new column alongside it.

EXAMPLE CARD

Side-by-side: GROUP BY vs window function on the exact same question - 'what is the average salary per department?'

sql - Code Example

```
-- GROUP BY: collapses every employee into one row per department
SELECT department,
       AVG(salary) AS avg_dept_salary
FROM   employees
GROUP BY department;
```

Output - **3 rows total** (one per department), individual employees are gone:

code - Code Example

department	avg_dept_salary
Engineering	95000.00
Sales	72000.00
Marketing	68000.00

sql – Code Example

```
-- WINDOW FUNCTION: every employee row survives, with the average
attached
SELECT employee_id,
       name,
       department,
       salary,
       AVG(salary) OVER (PARTITION BY department) AS avg_dept_salary
FROM   employees;
```

Output - **every employee row still present** (say 8 rows for 8 employees), each carrying their department's average as an extra column:

code – Code Example

employee_id	name	department	salary	avg_dept_salary
1	Alice	Engineering	98000	95000.00
2	Bob	Engineering	92000	95000.00
3	Carol	Sales	75000	72000.00
4	Dave	Sales	69000	72000.00
5	Eve	Marketing	68000	68000.00

Notice: Alice and Bob both show **avg_dept_salary = 95000.00** - the SAME computed value - but Alice's own row and Bob's own row are **both still there**, each with their own individual salary intact. This is impossible to get from a plain GROUP BY without a self-join or subquery.

🔗 ANALOGY

GROUP BY is like asking everyone in a department to merge into one spokesperson who reports the team average and nobody else speaks again. A window function is like asking everyone to stay exactly where they are, but handing each person a sticky note showing their team's average - every individual is still visible, just with extra context attached.

CONCEPT CARD

The OVER() clause is the heart of every window function. Without it, a function like RANK() or AVG() has no meaning - OVER() tells the database what 'window' of rows to compute across for each row. It has two key sub-clauses:

sql – Code Example

```
function_name( ... ) OVER (  
    PARTITION BY column_a    -- divides rows into groups, like GROUP  
    BY  
                                -- BUT without collapsing them  
    ORDER BY column_b        -- orders rows WITHIN each partition  
                                -- required for ranking functions to  
mean anything  
)
```

- **PARTITION BY** - resets the window function's calculation for each distinct value of the partitioning column(s). Within **PARTITION BY department**, the calculation restarts fresh for Engineering, restarts fresh for Sales, restarts fresh for Marketing. Omitting PARTITION BY treats the entire result set as one single partition.
- **ORDER BY** - establishes the order rows are processed in within each partition. For ranking functions, this ORDER BY is not optional cosmetic sorting - it directly determines what '1st', '2nd', '3rd' mean.

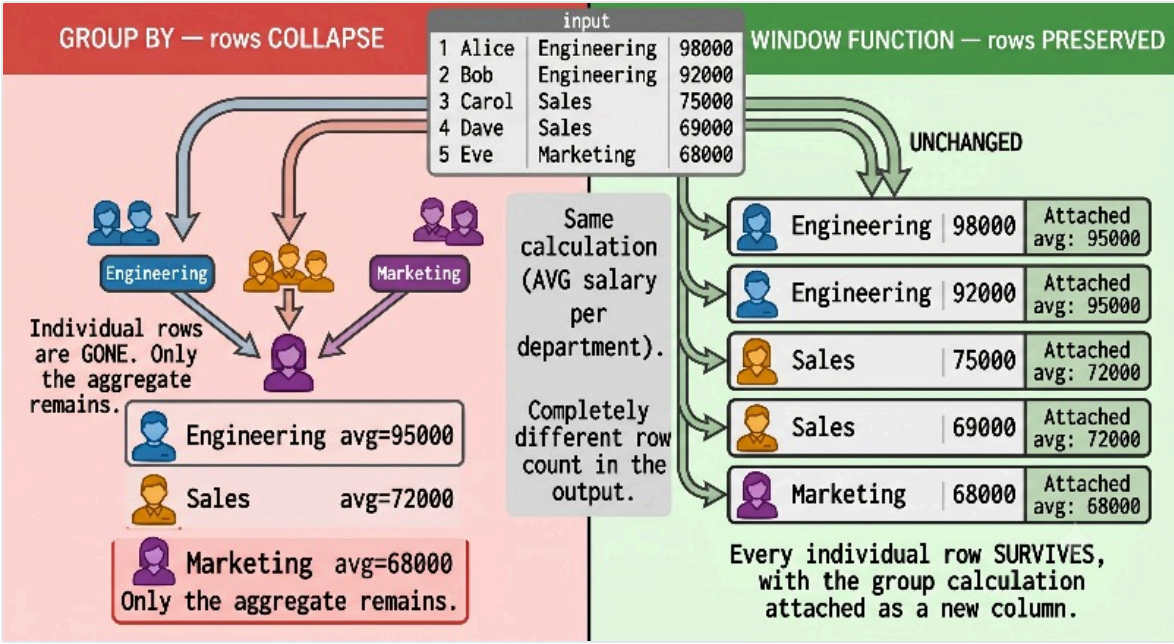
Student

What schema are we using?

ChatGPT • AI Assistant

CONCEPT CARD

employees(employee_id, name, department, salary) for the ranking examples, and the familiar e-commerce schema - **orders(order_id, customer_id, order_date, total_amount, status)** - for the order-sequencing practice problem.



GROUP BY Collapses Rows vs Window Function Preserves Rows

ROW_NUMBER — UNIQUE SEQUENTIAL NUMBERS, NO TIES EVER

ChatGPT • AI Assistant

A single-partition diagram showing the Engineering department's 4 employees stacked vertically in salary-descending order. Row 1 (gold, label 'Alice — salary=98000 — row_num=1'). Row 2 (blue, label 'Bob — salary=92000 — row_num=2'). Row 3 (blue, same shade as Bob to show they are tied in VALUE, label 'Carol — salary=92000 — row_num=3'). A bracket connects Bob and Carol's rows with a red annotation: 'TIED on salary (both 92000) — but ROW_NUMBER still assigns DIFFERENT numbers (2 and 3). No gap, no shared number — ever.' Row 4 (gray, label 'Dave — salary=85000 — row_num=4'). A side panel shows a warning icon with text: 'Without a tiebreaker column, whether Bob gets row_num=2 or Carol gets row_num=2 is NOT guaranteed to be consistent across runs.' A fix panel below shows the corrected ORDER BY: 'ORDER BY salary DESC, employee_id ASC — now Bob (lower employee_id) always wins the tie, deterministically.' Clean vertical stack, tied rows visually linked with a bracket, warning and fix call-outs clearly separated.

ROW_NUMBER — Strict Sequential Numbering Despite Tied Values

Student

Let's start with the simplest ranking function. What does ROW_NUMBER actually do?

ChatGPT • AI Assistant

CONCEPT CARD

ROW_NUMBER() assigns a **unique, sequential integer** to every row within its partition, based on the ORDER BY - starting at 1 and incrementing by exactly 1 for every row, with **no ties ever**. If two rows have the exact same value in the ORDER BY column, ROW_NUMBER still assigns them two different numbers - the choice of which one gets the lower number is **arbitrary** unless you add a tiebreaker column.

sql – Code Example

```
SELECT employee_id,
       name,
       department,
       salary,
       ROW_NUMBER() OVER (
           PARTITION BY department
           ORDER BY salary DESC
       ) AS row_num
FROM   employees;
```

EXAMPLE CARD

Worked example - ranking employees by salary within department, including a tie:

Sample data (note Bob and Carol are tied at 92000 in Engineering):

code – Code Example

employee_id	name	department	salary
1	Alice	Engineering	98000
2	Bob	Engineering	92000
3	Carol	Engineering	92000
4	Dave	Engineering	85000
5	Eve	Sales	75000
6	Frank	Sales	75000
7	Grace	Sales	60000

Output of **ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC)** :

code – Code Example

name	department	salary	row_num
Alice	Engineering	98000	1
Bob	Engineering	92000	2
Carol	Engineering	92000	2

← tied with Carol, but gets 2

Carol	Engineering	92000	3	← tied with Bob, but gets 3
Dave	Engineering	85000	4	
Eve	Sales	75000	1	
Frank	Sales	75000	2	← tied with Eve, but gets 2
Grace	Sales	60000	3	

Bob and Carol are both paid exactly 92000, yet *ROWNUMBER* hands them 2 and 3 respectively - sequential numbers with no gap and no tie, purely because *ROWNUMBER*'s entire job is to produce a strict 1-2-3-4 sequence regardless of value ties.

⚠ WARNING

Without a tiebreaker in ORDER BY, which tied row gets the lower number is not guaranteed to be consistent across different runs of the same query, especially after data changes, index rebuilds, or on different database versions. If you genuinely need deterministic, repeatable results for tied rows, add a secondary ORDER BY column:

sql – Code Example

```
ROW_NUMBER() OVER (  
    PARTITION BY department  
    ORDER BY salary DESC, employee_id ASC -- tiebreaker: lower id  
    wins ties  
) AS row_num
```

🧑 INTERVIEW CARD

Frequency: Very High

Question: What does ROW_NUMBER() do with tied values, and why might that be a problem?

Expected Answer: ROW_NUMBER() assigns a strictly sequential integer to every row in a partition with no duplicates ever - even rows with identical ORDER BY values receive different, consecutive numbers. This can be a problem because the choice of which tied row receives the lower number is arbitrary and not guaranteed to be stable across query re-runs unless an explicit tiebreaker column is added to the ORDER BY clause.

For applications where deterministic output matters - like consistently picking 'the' top employee when two are tied - you must add a secondary sort key, commonly a primary key column, to guarantee the same row wins every time.

Follow-up: If you used `ROWNUMBER()` to pick 'the single top earner per department' and two employees are tied for first place, what happens? (Answer: `ROWNUMBER()` will assign `rownum = 1` to only ONE of the two tied employees - arbitrarily, unless a tiebreaker is specified - meaning the other equally-deserving employee is excluded entirely if you filter `WHERE rownum = 1`.)

This is a common bug; `RANK()` or `DENSE_RANK()` would correctly show both as rank 1 if true ties should be reflected.)

🔗 PRACTICE QUIZ

1. Does `ROW_NUMBER()` ever assign the same number to two rows in the same partition?
2. What determines which tied row gets the lower `ROW_NUMBER` when there is no tiebreaker?
3. How do you make `ROW_NUMBER`'s tie-breaking behavior deterministic and repeatable?
4. In the worked example, what `row_num` does Dave receive and why?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. No - never. `ROW_NUMBER()` always produces a strict, gapless 1-2-3-4... sequence with zero duplicate numbers within a partition, regardless of value ties.
2. It is arbitrary/undefined by the database - unless an explicit tiebreaker column is added to `ORDER BY`, the order among tied rows is not guaranteed to be consistent.
3. Add a secondary column to the `ORDER BY` clause within `OVER()` - e.g., `ORDER BY salary DESC, employee_id ASC` - so that ties on salary are broken consistently by `employee_id` every time.
4. `row_num = 4` - Dave's salary (85000) is strictly less than Alice (98000, rank 1) and both Bob and Carol (92000, ranks 2 and 3), so Dave is simply the 4th row in the Engineering partition's salary-descending order.

🔗 CHECKPOINT QUESTION

Write a query using `ROWNUMBER()` that assigns a sequence number to every employee within their department, ordered by salary descending, with `employeeid`

ascending as a tiebreaker. Then explain what would change in the output if you removed the tiebreaker and ran the exact same query twice in a row.

RANK — TIED ROWS SHARE A RANK, THEN A GAP FOLLOWS

ChatGPT • AI Assistant

A single-partition diagram showing the same 4 Engineering employees from the ROW_NUMBER figure, now with RANK() values. Row 1 (gold, label 'Alice — salary=98000 — rnk=1'). Row 2 (blue, label 'Bob — salary=92000 — rnk=2'). Row 3 (blue, same shade, label 'Carol — salary=92000 — rnk=2') — connected to Bob's row by a bracket with green annotation: 'TIED — both correctly receive rank 2.' Row 4 (gray, label 'Dave — salary=85000 — rnk=4') with a prominent red gap-marker between Carol's row and Dave's row showing a crossed-out 'rank 3' label and annotation: 'Rank 3 is SKIPPED — never assigned to anyone. The gap size (1) equals the number of EXTRA rows that tied (2 rows tied, so 1 number is skipped).' A formula box at the bottom reads: 'Gap size after a tie = (number of tied rows) - 1.' Clean vertical stack matching the ROW_NUMBER figure's layout for direct visual comparison, with the skipped rank explicitly crossed out and labeled.

RANK — Tied Rows Share a Number, Then a Gap Appears

Student

Now show me RANK on the exact same tied data so I can see how it differs from ROW_NUMBER.

ChatGPT • AI Assistant

CONCEPT CARD

RANK() gives **tied rows the exact same rank**, but then **skips numbers** for the rows that follow - leaving a visible gap in the sequence equal to the number of rows that tied.

sql — Code Example

```
SELECT employee_id,
       name,
       department,
       salary,
       RANK() OVER (
           PARTITION BY department
           ORDER BY salary DESC
       ) AS rnk
FROM   employees;
```

EXAMPLE CARD

Same dataset as before, now with RANK():

code – Code Example

name	department	salary	rnk	
Alice	Engineering	98000	1	
Bob	Engineering	92000	2	← tied with Carol
Carol	Engineering	92000	2	← SAME rank as Bob
Dave	Engineering	85000	4	← jumps from 2 straight to 4!
Eve	Sales	75000	1	
Frank	Sales	75000	1	← SAME rank as Eve
Grace	Sales	60000	3	← jumps from 1 straight to 3!

Bob and Carol both correctly receive **rank 2**, reflecting that they are genuinely tied. But notice Dave: he is the 4th row by position, and even though only two distinct salary values came before him (98000 and 92000), Dave's rank is **4, not 3** - RANK() reserves rank 3 as if it were used, then skips it, because two rows occupied rank 2.

The gap size always equals the number of rows that tied at the rank just above.

Same thing happens in Sales: Eve and Frank tie at rank 1, and Grace - who is genuinely the 3rd row, but only the 2nd distinct salary value - receives **rank 3**, not rank 2.

🔗 ANALOGY

Think of RANK() like an Olympic medal podium where ties are allowed: if two athletes tie for gold, they are BOTH awarded 'rank 1' - but then there is officially no 'rank 2' awarded that event, and the next athlete is recorded as 'rank 3', because two people already hold the position that rank 2 would have occupied.

⚠️ WARNING

RANK() gaps can break naive 'top N' filtering if you are not careful. If you write `WHERE rnk ≤ 2` expecting exactly 2 rows per department, and there is a tie at rank 1, you will actually get **3 or more rows** for that department - both rank-1 rows plus possibly more, since rank 2 might not even exist if 2+ rows tied at rank 1.

sql – Code Example

```
-- If Bob and Carol BOTH tie at rank 1 instead of 2 in this example,
-- WHERE rnk ≤ 2 would return Alice (no – wait, Alice is highest)
```

```
-- Using the Sales tie as illustration: Eve and Frank both = rank 1.  
-- WHERE rnk ≤ 2 on Sales returns: Eve (rnk=1), Frank (rnk=1), Grace  
(rnk=3)?  
-- NO – Grace is rnk=3, so WHERE rnk ≤ 2 correctly returns just Eve  
and Frank.  
-- But if you wanted EXACTLY 2 rows and got 2 due to the tie, that's a  
coincidence  
-- of this specific data – a 3-way tie at rank 1 would still only  
return  
-- those 3 tied rows under ≤ 2, since there's no rank 2 row to also  
include.
```

The real risk: assuming `WHERE rnk ≤ N` always returns exactly N rows per partition. With ties, it can return **more than N** rows (all the tied rows sharing a rank within your cutoff) or **fewer** if a rank value you expected to exist was skipped over.

INTERVIEW CARD

Frequency: Very High

Question: Explain exactly how RANK() handles tied values, including what happens to the rank number immediately after a tie.

Expected Answer: RANK() assigns identical rank values to rows that tie on the ORDER BY column(s) within their partition. Critically, it then skips the rank numbers that the tied rows would have otherwise occupied individually - if two rows tie at rank 2, the very next distinct row receives rank 4, not rank 3, because rank 3 is treated as already 'used up' by the second of the two tied rows.

The size of the gap after a tie always equals the number of rows involved in that tie minus one. This means RANK() values are not necessarily a clean contiguous sequence, and filtering with `WHERE rnk ≤ N` can return more or fewer than N rows per partition depending on where ties occur.

Follow-up: If three rows tie at rank 1, what rank does the next distinct row receive? (Answer: Rank 4 - the three tied rows consume positions 1, 2, and 3 collectively (all reported as rank 1), so the next distinct value starts at rank 4, skipping ranks 2 and 3 entirely.)

PRACTICE QUIZ

1. If two rows tie at rank 3, what rank does the next row receive?
2. In the Sales example, why does Grace receive rank 3 and not rank 2?
3. Does RANK() ever produce a gap in the rank sequence if there are no ties at all in the partition?

4. Why can `WHERE  $\text{rnk} \leq 2$` sometimes return more than 2 rows per partition?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

- 1. Rank 5 - the two tied rows occupy ranks 3 and 4 collectively (both reported as 3), so the next distinct value skips to 5.
- 2. Because Eve and Frank both tie at rank 1, consuming ranks 1 and 2 collectively. Grace is the next distinct salary value, so she receives rank 3 - the rank that would have been '2' in a no-ties world is skipped.
- 3. No - with zero ties, `RANK()` produces the exact same clean 1, 2, 3, 4... sequence as `ROW_NUMBER()`, since there is nothing to cause a gap.
- 4. Because if multiple rows tie at rank 1 (or any rank ≤ 2), all of them satisfy `$\text{rnk} \leq 2$` and are returned - a 2-way tie at rank 1 plus a row at rank 3 would return only the 2 tied rows under `$\leq 2$` (correctly 2 rows), but a 3-way tie at rank 1 would return all 3 tied rows under `$\leq 2$` , since all three share a rank value that is ≤ 2 .

🔍 CHECKPOINT QUESTION

Four employees in the Marketing department all have the identical salary of 70000, and a fifth has 65000. Write out by hand what `RANK()` would assign to all five rows ordered by salary `DESC`, and state exactly which rank value (if any) is skipped.

DENSE_RANK — TIED ROWS SHARE A RANK, NO GAPS AT ALL

ChatGPT • AI Assistant

A single wide comparison table diagram rendered as a visual grid, showing the 4 Engineering employees as rows and three column groups: ROW_NUMBER, RANK, DENSE_RANK. Each employee row shows all three values simultaneously: Alice (1, 1, 1) all matching since no tie yet. Bob and Carol's two rows are bracket-linked across all three columns showing (2,3) for ROW_NUMBER in blue with annotation 'different', (2,2) for RANK in green with annotation 'same — tied correctly', (2,2) for DENSE_RANK in green with annotation 'same — tied correctly'. Dave's row shows a stark three-way contrast: ROW_NUMBER=4, RANK=4, DENSE_RANK=3 — with the RANK and ROW_NUMBER values circled together in orange labeled 'both reflect raw POSITION (4th row)' and DENSE_RANK circled separately in purple labeled 'reflects distinct VALUE count (3rd distinct salary level)'. A summary strip beneath the table reads: 'ROW_NUMBER: never ties. RANK: ties + gaps (reflects position). DENSE_RANK: ties + no gaps (reflects distinct value count)'. Clean grid-style comparison, consistent color coding per function across the whole diagram for easy visual scanning.

All Three Ranking Functions Side by Side on Identical Tied Data

And the third one - DENSE_RANK. How is it different from RANK?

ChatGPT • AI Assistant

CONCEPT CARD

DENSE_RANK() also gives tied rows the **same rank**, exactly like RANK() - but it **never skips a number** afterward. The next distinct value simply gets the next consecutive integer, regardless of how many rows tied before it.

sql – Code Example

```
SELECT employee_id,
       name,
       department,
       salary,
       DENSE_RANK() OVER (
           PARTITION BY department
           ORDER BY salary DESC
       ) AS dense_rnk
FROM   employees;
```

EXAMPLE CARD

All three functions side by side, on the identical tied dataset - the difference is now unmistakable:

code – Code Example

name	department	salary	row_num	rnk	dense_rnk	
Alice	Engineering	98000	1	1	1	
Bob	Engineering	92000	2	2	2	
Carol	Engineering	92000	3	2	2	
Dave	Engineering	85000	4	4	3	← compare rnk(4) vs dense_rnk(3)
Eve	Sales	75000	1	1	1	
Frank	Sales	75000	2	1	1	
Grace	Sales	60000	3	3	2	← compare rnk(3) vs dense_rnk(2)

Reading this table is the single best way to internalise the difference:

- **ROW_NUMBER** never repeats a number, ever - Bob=2, Carol=3 even though tied.

- **RANK** repeats numbers for ties (Bob=2, Carol=2) but then skips ahead (Dave=4, not 3) - the rank value reflects *position*, including the 'lost' positions consumed by ties.
- **DENSE_RANK** repeats numbers for ties (Bob=2, Carol=2) and does NOT skip ahead (Dave=3) - the rank value reflects the count of *distinct* values seen so far, completely ignoring how many rows shared each value.

ANALOGY

If RANK() is the Olympic podium that 'wastes' a position on a tie, DENSE_RANK() is more like counting 'how many different prize tiers exist so far' - gold, gold, silver, bronze becomes tier 1, tier 1, tier 2, tier 3 - there is no missing tier 2 just because two athletes happened to tie for gold.

CONCEPT CARD

When to choose which function - the decision rule:

code – Code Example

Use ROW_NUMBER() when:

- You need a guaranteed unique row identifier per partition (e.g., pagination, "give me exactly 1 row representing this group")
- Ties should be broken arbitrarily or by an explicit tiebreaker
- You are building a sequence number for display or pagination purposes

Use RANK() when:

- You want competition-style ranking where ties are visible AND the position numbers reflect TRUE statistical position (e.g., "this salary is higher than 90% of others" type reasoning)
- Gaps after ties are semantically meaningful to you

Use DENSE_RANK() when:

- You want competition-style ranking where ties are visible, but you care about the COUNT of distinct tiers/levels, not raw position
- Common in tier/bucket-style reporting: "how many distinct salary levels exist in this department, and which level is each employee in"

INTERVIEW CARD

Frequency: Very High

Question: Walk me through the difference between `ROWNUMBER`, `RANK`, and `DENSERANK` using a dataset with a tie.

Expected Answer: All three assign a rank-like integer based on `ORDER BY` within a partition, but they handle ties completely differently. `ROW_NUMBER` never produces a tie in its output - even rows with identical `ORDER BY` values get distinct, sequential numbers, with the order among ties being arbitrary unless a tiebreaker is specified. `RANK` gives tied rows the identical rank value, but then skips ahead in the numbering for subsequent rows by the count of tied rows minus one - so a 2-way tie at rank 2 is followed by rank 4, not rank 3. *`DENSERANK` also gives tied rows the identical rank value, but never skips any numbers afterward - the same 2-way tie at rank 2 is followed by rank 3, because `DENSERANK` counts the number of distinct values seen so far, not raw row position.*

Follow-up: If you needed to know 'how many distinct salary tiers exist in this department and which tier each employee falls into', which function would you use and why? (Answer: `DENSE_RANK` - because it produces a clean, gapless count of distinct value tiers, which directly answers 'how many different levels exist' without the artificial gaps that `RANK` introduces purely due to how many rows happened to tie at each level.)

🔍 PRACTICE QUIZ

1. In the side-by-side table, why does Dave receive `rnk=4` but `dense_rnk=3`?
2. Which of the three functions would you use to implement 'show me page 2 of search results, 10 rows per page'?
3. If there are zero ties anywhere in a partition, do `RANK` and `DENSE_RANK` produce different output from each other?
4. What does `DENSE_RANK`'s value represent conceptually, in one sentence?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Because `RANK()` counts raw row positions including the ones 'consumed' by the Bob/Carol tie (so Dave is the 4th row position), while `DENSE_RANK()` counts only the number of `DISTINCT` salary values seen so far (98000, 92000, 85000 = 3 distinct values by the time we reach Dave), completely ignoring that 92000 happened to belong to two rows.

2. `ROWNUMBER()` - pagination requires a strictly unique, gapless sequence number per row so you can reliably compute `'WHERE rownum BETWEEN 11 AND 20'` for page 2.
3. No - with zero ties in a partition, `RANK()` and `DENSE_RANK()` produce identical output to each other (and to `ROWNUMBER()`), since there are no ties to cause either skipping or tier-counting behavior to diverge.
4. `DENSE_RANK`'s value represents the count of distinct `ORDER BY` values encountered so far within the partition, with all rows sharing a value receiving that same count.

🔍 CHECKPOINT QUESTION

Using the same Marketing department scenario from the previous checkpoint (four employees tied at 70000, one at 65000), write out by hand what `DENSE_RANK()` would assign to all five rows, and explicitly state how this differs from the `RANK()` values you computed earlier.

TOP-N-PER-GROUP WITH ROW_NUMBER, AND WHY WINDOW FUNCTIONS CANNOT GO IN WHERE

ChatGPT • AI Assistant

A vertical flowchart showing SQL's logical execution order as four stacked stages with a window function timing conflict highlighted. Stage 1 (top, orange, label 'FROM — tables identified'). Arrow down. Stage 2 (red, label 'WHERE — rows filtered'). A red X icon sits next to this stage with annotation: 'Window function values DO NOT EXIST YET at this point — nothing to compare against.' Arrow down. Stage 3 (blue, label 'SELECT — window functions COMPUTED HERE') — this stage is highlighted with a glowing border and a green checkmark, annotation: 'ROW_NUMBER(), RANK(), DENSE_RANK() all execute here, AFTER WHERE has already run.' Arrow down. Stage 4 (purple, label 'ORDER BY — window function results ARE available here'). Below the flowchart, a two-panel code comparison shows: Left panel (red border, label 'FAILS') — query with `RANK() OVER(...)` written directly inside `WHERE`, with a red strike-through. Right panel (green border, label 'WORKS') — a two-level structure showing a CTE computing `RANK()` in its `SELECT`, with the outer query's `WHERE` clause filtering the CTE's already-computed `rnk` column, labeled 'WHERE here filters a REGULAR COLUMN from the CTE's output — no conflict.' Clean vertical flowchart with the conflict stage clearly marked, code comparison panels directly below for the fix.

Why Window Functions Cannot Go in WHERE — Execution Order Conflict

Student

This is the part I actually need most often - finding the top 3 highest earners in each department. How do I use what we just learned for that?

ChatGPT • AI Assistant

CONCEPT CARD

This is the single most common real-world use of ranking window functions: **'top N per group.'** The pattern is always the same - compute the rank inside a CTE (or subquery), then filter on that rank in the outer query.

sql – Code Example

```
WITH ranked_employees AS (  
    SELECT employee_id,  
           name,  
           department,  
           salary,  
           ROW_NUMBER() OVER (  
               PARTITION BY department  
               ORDER BY salary DESC, employee_id ASC -- tiebreaker  
           ) AS rn  
    FROM employees  
)  
SELECT employee_id, name, department, salary, rn  
FROM ranked_employees  
WHERE rn ≤ 3  
ORDER BY department, rn;
```

This returns exactly the top 3 highest-paid employees per department, with a clean $rn = 1, 2, 3$ for every department regardless of ties elsewhere in the data, because the tiebreaker in ORDER BY guarantees ROW_NUMBER never produces duplicate values within a partition.

Student

Why do I need a CTE here? Why can't I just write `WHERE ROW_NUMBER() OVER (... ) ≤ 3` directly in the main query?

ChatGPT • AI Assistant

⚠ WARNING

This is one of the most common syntax errors beginners hit with window functions - and the reason is purely about SQL's logical execution order, which you learned back in SQL 3.1:

sql – Code Example

```
-- THIS FAILS:
SELECT employee_id, name, department, salary,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary
DESC) AS rn
FROM   employees
WHERE  ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary
DESC) ≤ 3;
-- ERROR: window functions are not allowed in WHERE
```

Why: recall the logical execution order - **FROM → WHERE → SELECT → ORDER BY** . Window functions are computed as part of the **SELECT** step, which runs **after** WHERE.

At the point WHERE is being evaluated, the window function's result does not exist yet - there is nothing for WHERE to compare against. This is structurally identical to the SQL 3.1 rule that you cannot reference a SELECT alias inside WHERE, just applied to window functions instead of simple expressions.

code – Code Example

```
Logical execution order recap:
1. FROM      ← tables identified
2. WHERE     ← rows filtered ... window functions NOT available yet
3. SELECT    ← window functions computed HERE
4. ORDER BY  ← window function results ARE available here
```

The fix is always the same shape: **compute the window function in one query level (a CTE or subquery), then filter on its result in the level above.**

sql – Code Example

```
-- CORRECT pattern:
WITH ranked AS (
    SELECT ..., ROW_NUMBER() OVER (...) AS rn FROM employees
)
SELECT * FROM ranked WHERE rn ≤ 3;
-- The WHERE here filters the OUTPUT of the CTE, where rn already exists
-- as a regular column – this WHERE has nothing to do with the window
-- function's own internal computation, which already finished inside
the CTE.
```

CONCEPT CARD

The same top-N pattern works identically with RANK() if you want ties included:

sql – Code Example

```
WITH ranked_employees AS (  
    SELECT employee_id, name, department, salary,  
           RANK() OVER (PARTITION BY department ORDER BY salary DESC)  
    AS rnk  
    FROM    employees  
)  
SELECT * FROM ranked_employees  
WHERE  rnk ≤ 3  
ORDER BY department, rnk;  
-- If 2 employees tie for 2nd place, BOTH are included – you may get  
-- 4 rows for a department instead of exactly 3, because the tie is  
-- being honestly represented rather than arbitrarily broken.
```

Choosing *ROWNUMBER* vs *RANK* for 'top N' is itself a business decision: do you want ****exactly N rows**** per group no matter what (use *ROWNUMBER* with a tiebreaker), or do you want to **honestly represent ties**, even if that means occasionally returning more than N rows (use *RANK*)?

 INTERVIEW CARD

Frequency: Very High

Question: Why can't you put a window function directly inside a WHERE clause, and how do you work around this?

Expected Answer: Window functions are computed at the SELECT step of SQL's logical execution order, which runs after WHERE. At the moment WHERE is evaluated, the window function has not been computed yet, so there is nothing for the WHERE condition to reference - this produces a syntax/semantic error, structurally identical to why a SELECT alias cannot be referenced in the same query's WHERE clause.

The standard workaround is to compute the window function inside a CTE or subquery, then filter on the resulting column name in the outer query's WHERE clause - at that point the window function's result already exists as an ordinary column from the inner query's perspective.

Follow-up: If you want exactly 3 rows per department for a top-3 report, but the data might have ties at the 3rd position, should you use *ROWNUMBER* or *RANK*? (Answer: *ROWNUMBER* with an explicit tiebreaker column in its ORDER BY - this guarantees exactly 3 rows per department regardless of ties, since ROW_NUMBER never produces duplicate values within a partition.

RANK would honestly show all tied rows at a given position, which could mean more than 3 rows are returned for departments with a tie at the cutoff.)

🔗 PRACTICE QUIZ

1. At which step of SQL's logical execution order are window functions computed?
2. Why does this error: `SELECT * FROM orders WHERE RANK() OVER (ORDER BY total_amount DESC) ≤ 5; ?`
3. What is the standard fix for filtering on a window function's result?
4. If you use RANK() instead of ROW_NUMBER() for a 'top 3' report, can the output ever contain more than 3 rows for a single partition?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. The SELECT step - window functions are part of SELECT's column computation, which runs after FROM and WHERE in the logical execution order.
2. Because WHERE is evaluated before SELECT, and the RANK() window function is computed during SELECT - at WHERE-evaluation time, the RANK() value does not exist yet to compare against.
3. Wrap the window function computation in a CTE (or subquery), giving it a column alias, then write the WHERE clause in the outer query referencing that alias as an ordinary column.
4. Yes - if multiple rows tie at exactly the rank-3 boundary (or any rank ≤ 3), all of them share that rank and all satisfy `WHERE rnk ≤ 3`, so the partition can return more than 3 rows total.

🔗 CHECKPOINT QUESTION

Write a complete CTE-based query that returns the top 2 most expensive orders (by *totalamount*) *per customer*, using ROWNUMBER with *orderid* as a *tiebreaker*. Then explain what would change in the result if you swapped ROWNUMBER for DENSE_RANK in the exact same query.

ChatGPT • AI Assistant

A three-column decision comparison diagram showing the SAME Engineering department data (Alice and Bob tied at 98000, Carol at 92000, Dave at 85000) run through all three functions, with the goal 'find the 2nd highest salary' highlighted. Column 1 (red background, label 'ROW_NUMBER — WRONG for this goal'): shows Alice=rn1, Bob=rn2, Carol=rn3, Dave=rn4. A red X marks Bob's row with annotation: 'WHERE rn=2 returns BOB — but Bob earns the SAME as Alice (98000), not a genuinely lower salary! This is WRONG.' Column 2 (orange background, label 'RANK — ALSO WRONG'): shows Alice=rnk1, Bob=rnk1, Carol=rnk3, Dave=rnk4. An orange warning mark on the rnk=2 search with annotation: 'WHERE rnk=2 returns ZERO ROWS — rank 2 was skipped entirely because of the tie at rank 1. The true second-highest earner is invisible to this query.' Column 3 (green background, label 'DENSE_RANK — CORRECT'): shows Alice=dr1, Bob=dr1, Carol=dr2, Dave=dr3. A green checkmark on Carol's row with annotation: 'WHERE salary_rank=2 correctly returns CAROL — the true second-highest DISTINCT salary level, completely unaffected by the Alice/Bob tie at the top.' A bottom banner reads: 'Rule of thumb: "Nth highest VALUE" questions almost always want DENSE_RANK, not ROW_NUMBER or RANK.' Clean three-column comparison, consistent red/orange/green verdict coloring.

Choosing the Right Ranking Function for '2nd Highest Salary' with a Tie at 1st

Student

Before the practice problems - you mentioned NTILE briefly. What does that do?

ChatGPT • AI Assistant

CONCEPT CARD

NTILE(n) divides the rows in each partition into **n roughly equal-sized buckets**, numbered 1 through n, based on the ORDER BY. It is commonly used for quartiles (**NTILE(4)**), deciles (**NTILE(10)**), or any 'split into n groups' reporting need.

sql – Code Example

```
SELECT employee_id,
       name,
       salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile
FROM   employees;
```

With 12 employees and **NTILE(4)** , the rows are split into 4 buckets of 3 employees each - bucket 1 contains the 3 highest earners, bucket 4 contains the 3 lowest. If the row count does not divide evenly (say 10 employees into 4 buckets), NTILE distributes the remainder to the earliest buckets first - some buckets get 3 rows, others get 2, but never more than a 1-row difference between any two buckets.

sql – Code Example

```
-- 10 employees, NTILE(4): buckets sized 3, 3, 2, 2 (the extra rows go to bucket 1 and 2)
```

This is conceptually different from the ranking functions:

ROWNUMBER/RANK/DENSERANK answer 'what is this row's position or tier relative to others', while **NTILE** answers 'which equal-sized SLICE of the whole dataset does this row fall into.'

Student

Now let's do the two practice problems.

ChatGPT • AI Assistant

CONCEPT CARD

Practice Problem 1 - Find the 2nd highest-paid employee in each department, handling ties correctly.

The phrase 'handling ties correctly' is the key design decision here: do we want the 2nd-highest **distinct salary value** (use **DENSERANK**, *so a tie at 1st place still lets us find a true 2nd distinct level*), or do we want literally 'the row in the 2nd position' (use **ROWNUMBER** with a tiebreaker)? The more common business intent for '2nd highest salary' is the **distinct value** interpretation - so **DENSE_RANK** is usually the correct choice:

sql – Code Example

```
WITH salary_levels AS (  
    SELECT employee_id,  
           name,  
           department,  
           salary,  
           DENSE_RANK() OVER (  
               PARTITION BY department  
               ORDER BY salary DESC  
           ) AS salary_rank  
    FROM employees  
)  
SELECT employee_id, name, department, salary  
FROM salary_levels
```

```
WHERE salary_rank = 2
ORDER BY department;
```

Why DENSE_RANK and not RANK or ROW_NUMBER here:

code – Code Example

- If two employees TIE for 1st place in a department:
 - ROW_NUMBER would assign one of them rn=1 and the other rn=2 – WRONG, because the 'true' 2nd-highest-paid employee would then incorrectly be reported as whichever person merely lost the arbitrary tiebreak for 1st place, even though they earn the SAME amount as the actual top earner – not a genuinely lower salary at all.
 - RANK would assign both of them rank 1, then the NEXT distinct salary gets rank 3 (skipping 2) – so WHERE rnk = 2 would return ZERO rows for that department, even though there clearly IS a meaningful 'second salary tier' in the data.
 - DENSE_RANK assigns both of them rank 1, and the next distinct salary correctly gets rank 2 – WHERE salary_rank = 2 correctly finds the true second-highest DISTINCT salary level, including every employee who earns that amount.

Sample trace - Engineering department with a tie for 1st:

code – Code Example

name	salary	dense_rnk	
Alice	98000	1	
Bob	98000	1	← tied for 1st with Alice
Carol	92000	2	← correctly identified as 2nd-highest
Dave	85000	3	

WHERE salary_rank = 2 correctly returns Carol - the genuinely second-highest distinct salary in Engineering - regardless of the Alice/Bob tie at the top.

Practice Problem 2 - Assign every order a sequence number within each customer's order history, ordered by date.

sql – Code Example

```
SELECT order_id,
       customer_id,
       order_date,
       total_amount,
       ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date ASC, order_id ASC    -- tiebreaker for
same-day orders
       ) AS order_sequence
FROM   orders
ORDER BY customer_id, order_sequence;
```

Output (sample, for one customer):

code – Code Example				
order_id	customer_id	order_date	total_amount	order_sequence
5001	101	2024-01-05	850.00	1
← customer's FIRST ever order				
5034	101	2024-02-12	1200.00	2
← their SECOND order				
5089	101	2024-02-12	300.00	3
← same day as above, tiebreaker by order_id				
5210	101	2024-04-30	920.00	4

*ROWNUMBER is the correct choice here - not RANK or DENSERANK - because an order **sequence number is inherently about position**, not about tied values. Even two orders placed on the exact same date are still two genuinely distinct, sequential events in the customer's order history; there is no meaningful concept of 'tied' order sequence numbers the way there is a meaningful concept of tied salaries.*

The `order_id ASC` tiebreaker simply gives a stable, repeatable answer for same-day orders.

A common follow-up use of this exact pattern - flagging each customer's first order (useful for new-customer cohort analysis):

sql – Code Example	
<pre>WITH order_sequence AS (SELECT order_id, customer_id, order_date, total_amount, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date ASC,</pre>	

```

order_id ASC
        ) AS order_sequence
    FROM    orders
)
SELECT order_id, customer_id, order_date, total_amount
FROM    order_sequence
WHERE    order_sequence = 1;      -- exactly one row per customer: their
first order

```

SUMMARY CARD

SQL 1.13 Key Takeaways:

- **Window functions preserve every row;** GROUP BY collapses rows into one per group. This is the single defining difference.
- **OVER()** is required for every window function. **PARTITION BY** resets the calculation per group without collapsing rows. **ORDER BY** inside OVER() determines processing order and is required for ranking functions to be meaningful.
- **ROW_NUMBER()** - strictly sequential, never ties, arbitrary tie-breaking order unless you add a tiebreaker column.
- **RANK()** - ties share a rank, then the sequence SKIPS numbers afterward, reflecting raw row position.
- **DENSE_RANK()** - ties share a rank, NO numbers are skipped afterward, reflecting count of distinct values.
- **Top-N-per-group pattern:** compute the rank in a CTE, filter **WHERE rn ≤ N** in the outer query - never attempt to filter a window function directly in the same level's WHERE clause.
- **Window functions cannot appear in WHERE** because they are computed at the SELECT step, which runs after WHERE in SQL's logical execution order - same root cause as the SELECT-alias-in-WHERE restriction from SQL 3.1.
- **'2nd highest value' problems** almost always want **DENSERANK**, *not ROWNUMBER* or **RANK**, because the business question is usually about distinct value tiers, not raw row position.
- **NTILE(n)** splits a partition into n roughly equal-sized buckets - a different question entirely from ranking ('which slice am I in' vs 'what is my position/tier').

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned window function ranking - **ROWNUMBER**, **RANK**, **DENSERANK**, the **OVER/PARTITION BY/ORDER BY** anatomy, top-N-per-group

with a CTE, why window functions cannot go in WHERE, and NTILE for bucketing.

Give me 8 progressively harder exercises using realistic business scenarios (top customers per region, 3rd highest order per month, salary quartiles, deduplication using ROW_NUMBER). For each exercise: give me the table schema and a tied dataset, ask me to predict the output of all three ranking functions by hand BEFORE writing SQL, then ask me to write the query, then show the correct solution and one common mistake.

Do not continue until I submit my answer.'

🔍 CHECKPOINT QUESTION

Before moving to more advanced window functions (LAG, LEAD, and running totals in the next topic): a colleague wrote `SELECT *, ROW_NUMBER() OVER (ORDER BY order_date) AS rn FROM orders WHERE rn = 1;` to find each table's very first row, and it fails. Explain exactly why it fails using the logical execution order, and rewrite it correctly as a CTE-based query that finds the single earliest order overall (not per customer - across the whole table).